

**Université Abdelmalek Essaâdi**  
École Nationale des Sciences Appliquées  
Al Hoceima

## Rapport de TP5

Java SE

La généricité et la gestion des exceptions

|                              |                                       |
|------------------------------|---------------------------------------|
| <b>Réalisé par :</b>         | Mohamed Reda El Morabit               |
| <b>Filière :</b>             | 1ère année – Ingénierie Données (ID1) |
| <b>Encadré par :</b>         | Pr. Mohamed CHERRADI                  |
| <b>Année universitaire :</b> | 2025–2026                             |

# Partie 1 — Exceptions système (surveillées & non surveillées)

## Exercice 1 (Division sécurisée)

### Objectif

Écrire un programme qui permet de calculer la division de deux entiers en gérant la division par zéro et les saisies invalides avec try/catch et throw.

### Code

#### Classe EX1

```
1 package E;
2
3 import java.util.InputMismatchException;
4 import java.util.Scanner;
5
6 public class EX1 {
7
8     static class Calcul {
9         static int diviser(int a, int b) {
10             if (b == 0) throw new ArithmeticException("division par zero interdite");
11             return a / b;
12         }
13     }
14
15     public static void main(String[] args) {
16         Scanner sc = new Scanner(System.in);
17         System.out.println("entrez a :");
18         System.out.println("entrez b :");
19
20         try {
21             int a = sc.nextInt();
22             int b = sc.nextInt();
23
24             System.out.println(Calcul.diviser(a, b));
25
26         } catch (InputMismatchException e) {
27             System.out.println(" erreur : il faut entrer des entiers ");
28         } catch (ArithmeticException e) {
29             System.out.println(e.getMessage());
30         }
31     }
32 }
```

## Affichage

```
entrez a :  
entrez b :  
5  
0  
division par zero interdite
```

## Exercice 2 (NullPointerException)

### Objectif

Créer une méthode qui affiche la longueur d'une chaîne et tester le cas où elle est null en gérant l'exception sans et avec try/catch.

## Code

### Classe EX2

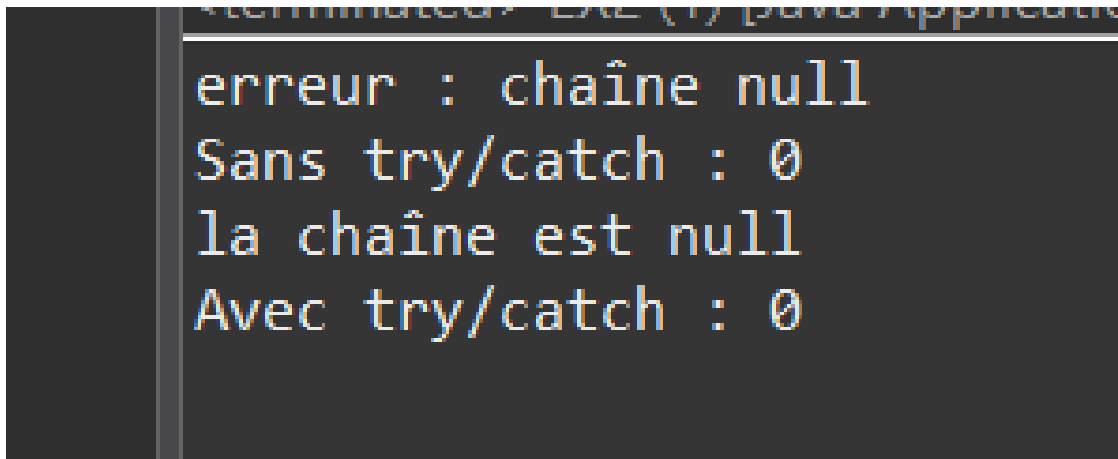
```
1 package E;  
2  
3 public class EX2 {  
4  
5     static class calcul {  
6  
7         static int LongueurSansEXC(String s) {  
8             if (s == null) {  
9                 System.out.println("erreur : chaîne null");  
10                return 0;  
11            }  
12            return s.length();  
13        }  
14  
15        static int LongueurAvecEXC(String s) {  
16            try {  
17                return s.length();  
18            } catch (NullPointerException e) {  
19                System.out.println("la chaîne est null");  
20                return 0;  
21            }  
22        }  
23    }  
24 }
```

```

22     }
23 }
24
25 public static void main(String[] args) {
26
27     String texte = null;
28
29     System.out.println("Sans try/catch : " +
30         calcul.LongueurSansEXC(texte));
31
32     System.out.println("Avec try/catch : " +
33         calcul.LongueurAvecEXC(texte));
34 }
35 }

```

## Affichage



```

C:\Program Files\Java\jre6\bin\java.exe
erreur : chaîne null
Sans try/catch : 0
la chaîne est null
Avec try/catch : 0

```

## Exercice 3 (Tableau sécurisé)

### Objectif

Créer un tableau et permettre l'accès à un index en gérant `ArrayIndexOutOfBoundsException` avec `if` et `try/catch`.

## Code

### Classe EX3

```

1 package E;
2
3 public class EX3 {
4
5     static class Tableau{
6
7         static int AvecIf(int[] tab, int i) {
8             if (i >= 0 && i < tab.length) {
9                 return tab[i];
10            } else {
11                System.out.println("erreur dindex hors limites");
12                return -1;
13            }
14        }
15
16        static int TryCatch(int[] tab, int i) {
17
18            try { return tab[i]; }
19            catch(ArrayIndexOutOfBoundsException e){
20                System.out.println("exception : index invalide (try/catch)");
21                return -1;
22            }
23        }
24    }
25
26    public static void main(String[] args) {
27
28        int[] tab = {9, 4, 3, 1, 0};
29
30        int indice = 6;
31
32        System.out.println("Valeur = " + Tableau.AvecIf(tab, indice));
33        System.out.println("Valeur = " + Tableau.TryCatch(tab, indice));
34
35    }
36
37 }
38
39
40

```

## Affichage

```

erreur dindex hors limites
Valeur = -1
exception : index invalide (try/catch)
Valeur = -1

```

## Exercice 4 (Conversion d'entier)

### Objectif

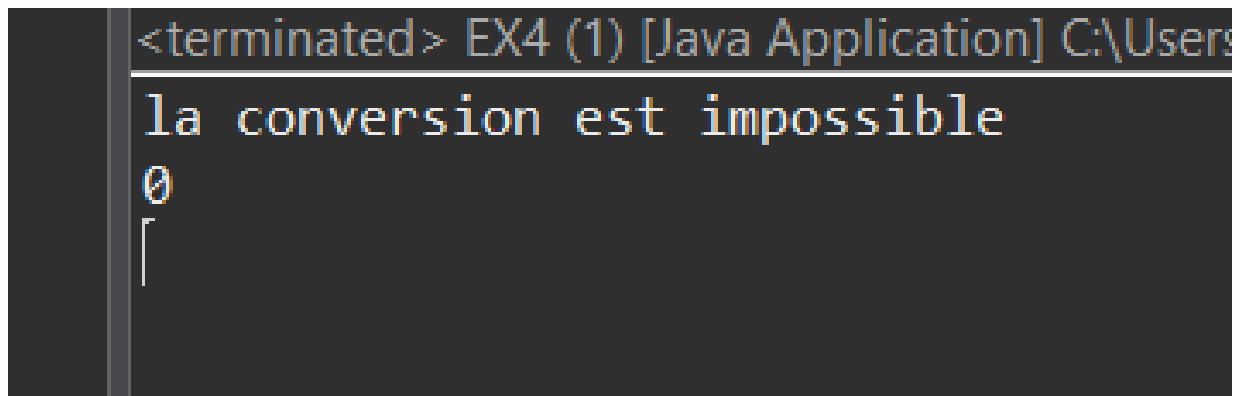
Convertir une chaîne en entier et gérer proprement `NumberFormatException`.

### Code

#### Classe EX4

```
1 package E;
2
3 public class EX4 {
4
5     static class calcul {
6         static int convertir(String s) {
7             try {
8                 return Integer.parseInt(s);
9             } catch (NumberFormatException e) {
10
11                 System.out.println("la conversion est impossible");
12                 return 0;
13             }
14         }
15     }
16 }
17
18 public static void main(String[] args) {
19     String S = "test";
20     System.out.println(
21         calcul.convertir(S)
22     );
23 }
24 }
```

### Affichage



```
<terminated> EX4 (1) [Java Application] C:\Users...
la conversion est impossible
0
|
```

## Exercice 5 (Choix d'exception)

### Objectif

Lever une exception appropriée (`IllegalArgumentException`) si on passe un entier négatif à une fonction.

### Code

#### Classe EX5

```
1 package E;
2
3 public class EX5 {
4
5     static class calcul {
6
7         static double racineCarree(int x) {
8
9             if (x < 0) {
10                 throw new IllegalArgumentException(
11                     "nombre negatif interdit"
12                 );
13             }
14             return Math.sqrt(x);
15         }
16     }
17     public static void main(String[] args) {
18
19         System.out.println(
20             calcul.racineCarree(1)
21         );
22         System.out.println(
23             calcul.racineCarree(-2)
24         );
25     }
26 }
```

### Affichage

```
1.0
Exception in thread "main" java.lang.IllegalArgumentException: nombre negatif interdit
    at TP5/E.EX5$calcul.racineCarree(EX5.java:10)
    at TP5/E.EX5.main(EX5.java:23)
```

## Exercice 6 (IllegalStateException)

### Objectif

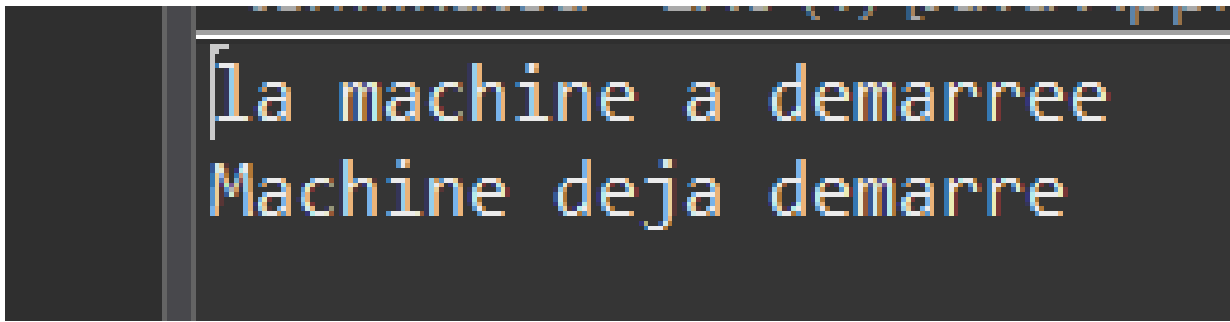
Créer une classe Machine et lancer IllegalStateException lors du démarrage si elle est déjà ON.

### Code

#### Classe EX6

```
1 package E;
2
3 public class EX6 {
4     static class Machine {
5         boolean etat = false;
6         void demarrer() {
7             if (etat) {
8                 throw new IllegalStateException(
9                     "Machine deja démarre"
10                );
11             }
12             etat = true;
13             System.out.println("la machine a démarree");
14         }
15     }
16
17     public static void main(String[] args) {
18
19         Machine m = new Machine();
20
21         try {
22
23             m.demarrer();
24             m.demarrer();
25
26         } catch (IllegalStateException e) {
27
28             System.out.println(e.getMessage());
29         }
30     }
31 }
```

## Affichage



```
la machine a demarree
Machine deja demarre
```

## Exercice 7 (Propagation)

### Objectif

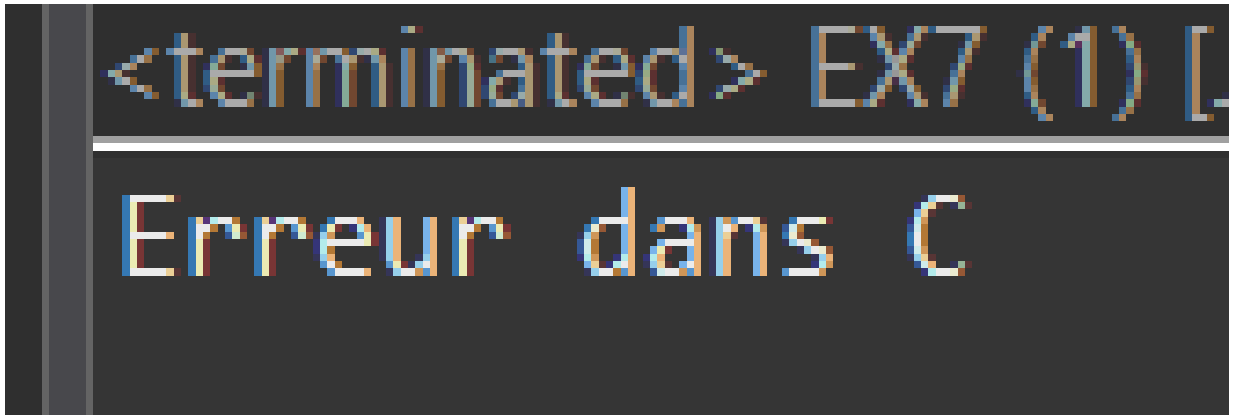
Démontrer la propagation des exceptions à travers l'appel de 3 méthodes (A gère l'exception lancée par C et propagée par B).

## Code

### Classe EX7

```
1 package E;
2
3 public class EX7 {
4     static class calcul {
5         static void C() {
6             throw new ArithmeticException("Erreur dans C");
7         }
8         static void B() {
9             C();
10        }
11        static void A() {
12            try {
13                B();
14            } catch (ArithmeticException e) {
15
16                System.out.println(e.getMessage());
17            }
18        }
19    }
20
21    public static void main(String[] args) {
22
23        calcul.A();
24    }
25 }
```

## Affichage



## Exercice 8 (Checked Exception)

### Objectif

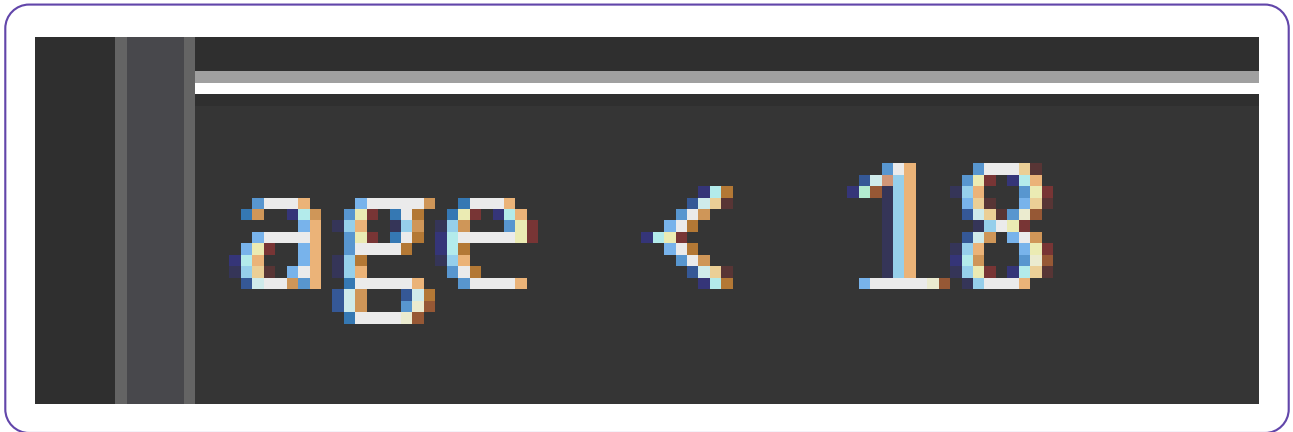
Créer une méthode avec throws Exception pour vérifier l'âge et gérer cette exception checked dans le main.

### Code

#### Classe EX8

```
1 package E;
2
3 public class EX8 {
4     static class calcul {
5         static void verifierAge(int age) throws Exception {
6             if (age < 18) {
7                 throw new Exception("age < 18");
8             }
9             System.out.println("age valide");
10        }
11    }
12    public static void main(String[] args) {
13        try {
14            calcul.verifierAge(15);
15        } catch (Exception e) {
16            System.out.println(e.getMessage());
17        }
18    }
19 }
```

## Affichage



# Partie 2 — Exceptions personnalisées (checked & unchecked)

## Exercice 1 (Compte bancaire)

### Objectif

Créer une exception personnalisée `SoldeInsuffisantException` et la lever lors d'un retrait excessif.

### Code

#### Classe EX1

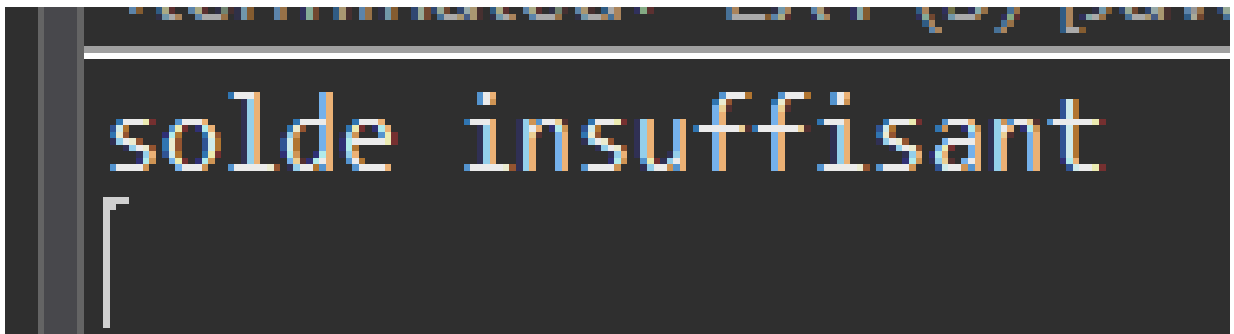
```
1 package E2;
2
3 class SoldeInsuffisantException extends Exception {
4
5     public SoldeInsuffisantException(String msg) {
6         super(msg);
7     }
8 }
9
10 public class EX1 {
11     static class CompteBancaire {
12         String code;
13         double solde;
14
15         CompteBancaire(String code, double solde) {
16             this.code = code;
17             this.solde = solde;
18         }
19         void verser(double montant) {
20             solde += montant;
21             System.out.println(
22                 "nouveau solde : " + solde
23             );
24         }
25
26         void retirer(double montant)
27             throws SoldeInsuffisantException {
28
29             if (montant > solde) {
30                 throw new SoldeInsuffisantException(
31                     "solde insuffisant"
32                 );
33             }
34
35             solde -= montant;
36
37             System.out.println(
38                 "nouveau solde : " + solde
```

```

39         );
40     }
41 }
42
43 public static void main(String[] args) {
44
45     CompteBancaire c =
46         new CompteBancaire("C1", 1590);
47
48     try {
49
50         c.retirer(2000);
51
52     } catch (SoldeInsuffisantException e) {
53
54         System.out.println(e.getMessage());
55     }
56 }
57 }

```

## Affichage



## Exercice 2 (Montant invalide)

### Objectif

Créer une exception MontantInvalideException pour bloquer un montant  $\leq 0$ .

### Code

#### Classe EX2

```

1 package E2;
2
3 class MontantInvalideException extends Exception {

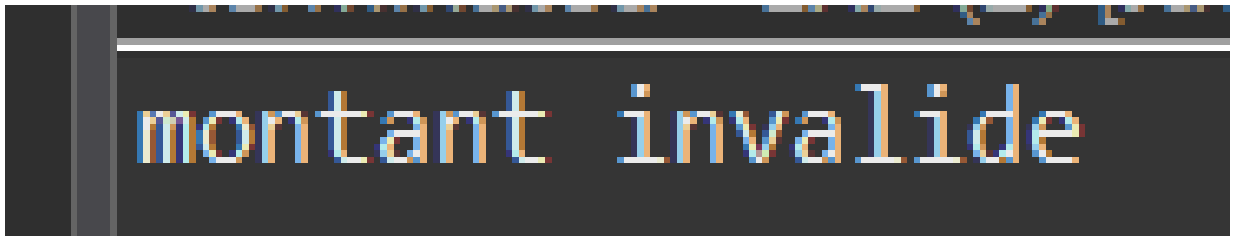
```

```

4     public MontantInvalideException(String msg) {
5         super(msg);
6     }
7 }
8
9 public class EX2 {
10     static void verifierMontant(double montant) throws MontantInvalideException {
11         if (montant <= 0) {
12             throw new MontantInvalideException("montant invalide (<= 0)");
13         }
14         System.out.println("Montant valide : " + montant);
15     }
16
17     public static void main(String[] args) {
18         try {
19             verifierMontant(-50);
20         } catch (MontantInvalideException e) {
21             System.out.println(e.getMessage());
22         }
23     }
24 }

```

## Affichage



## Exercice 3 (Double validation)

### Objectif

Modifier retirer() pour gérer conjointement un montant invalide et un solde insuffisant.

## Code

### Classe EX3

```

1 package E2;
2
3 class SoldeInsuffisantException extends Exception {
4     public SoldeInsuffisantException(String msg) { super(msg); }

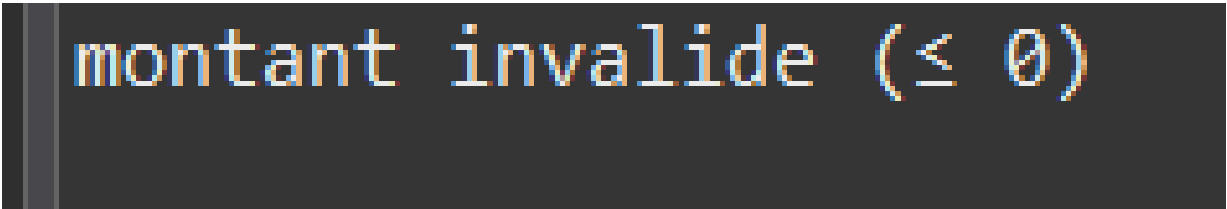
```

```

5 }
6
7 class MontantInvalideException extends Exception {
8     public MontantInvalideException(String msg) { super(msg); }
9 }
10
11 public class EX3 {
12     static class CompteBancaire {
13         double solde;
14         CompteBancaire(double solde) { this.solde = solde; }
15
16         void retirer(double montant) throws MontantInvalideException,
17             SoldeInsuffisantException {
18             if (montant <= 0) {
19                 throw new MontantInvalideException("Montant ≤ 0 impossible");
20             }
21             if (montant > solde) {
22                 throw new SoldeInsuffisantException("solde insuffisant");
23             }
24             solde -= montant;
25             System.out.println("Nouveau solde : " + solde);
26         }
27     }
28
29     public static void main(String[] args) {
30         CompteBancaire c = new CompteBancaire(500);
31         try {
32             c.retirer(600);
33         } catch (MontantInvalideException e) {
34             System.out.println(e.getMessage());
35         } catch (SoldeInsuffisantException e) {
36             System.out.println(e.getMessage());
37         }
38     }
39 }

```

## Affichage



```
montant invalide (<= 0)
```

## Exercice 4 (Inscription Utilisateur)

### Objectif

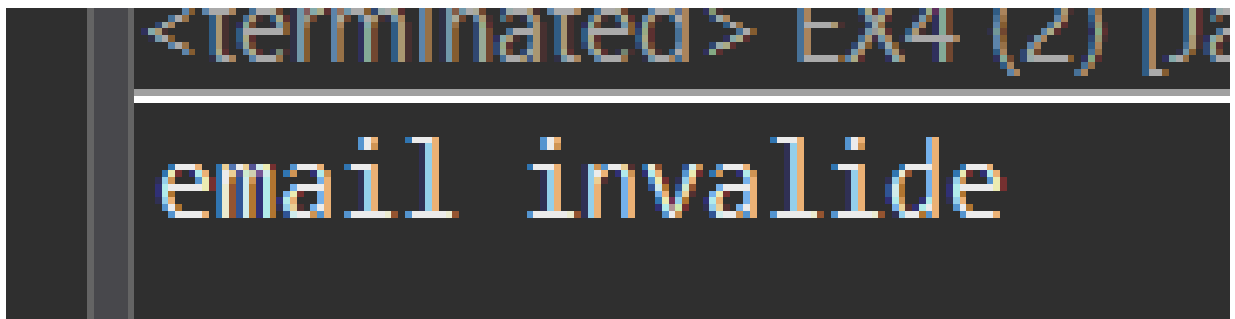
Créer et gérer les exceptions personnalisées `EmailInvalideException` et `AgeInvalideException`.

### Code

#### Classe EX4

```
1 package E2;
2
3 class EmailInvalideException extends Exception {}
4 class AgeInvalideException extends Exception {}
5
6 public class EX4 {
7     static void inscrire(String email, int age) {
8         try {
9             if (!email.contains("@")) throw new EmailInvalideException();
10            if (age < 18) throw new AgeInvalideException();
11            System.out.println("Inscription OK");
12        } catch (EmailInvalideException e) {
13            System.out.println("email invalide");
14        } catch (AgeInvalideException e) {
15            System.out.println("age <18");
16        }
17    }
18
19    public static void main(String[] args) {
20        inscrire("ab", 16);
21    }
22 }
```

### Affichage



The screenshot shows a terminal window with the output of the Java program. The text displayed is "email invalide". Above this, there is a line that says "<terminated> EX4 (2) [D".

## Exercice 5 (Authentication)

### Objectif

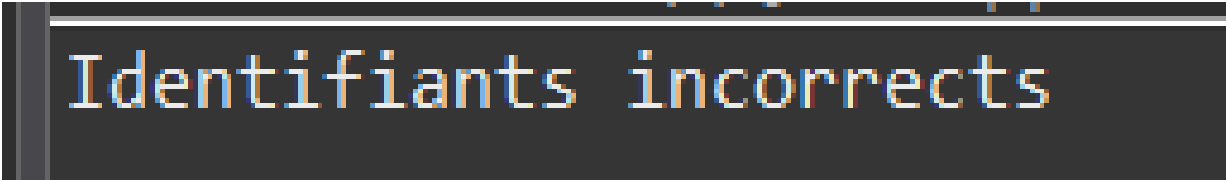
Lever une `AuthenticationException` en cas de mauvais identifiants.

### Code

#### Classe EX5

```
1 package E2;
2
3 class AuthenticationException extends Exception {}
4
5 public class EX5 {
6     static void login(String u, String p) {
7         try {
8             if (!"admin".equals(u) || !"123".equals(p)) throw new
                AuthenticationException();
9             System.out.println("Login OK");
10        } catch (AuthenticationException e) {
11            System.out.println("Identifiants incorrects");
12        }
13    }
14
15    public static void main(String[] args) {
16        login("x", "y");
17    }
18 }
```

### Affichage



## Exercice 6 (Gestion de stock)

### Objectif

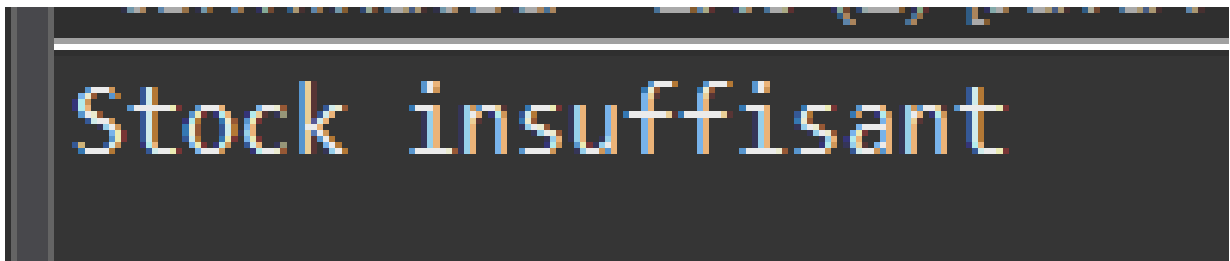
Gérer les retraits d'un produit avec `StockInsuffisantException` si la quantité demandée dépasse le stock.

## Code

### Classe EX6

```
1 package E2;
2
3 class StockInsufficientException extends Exception {}
4
5 public class EX6 {
6     static class Produit {
7         int stock = 5;
8         void retirer(int q) {
9             try {
10                 if (q > stock) throw new StockInsufficientException();
11                 stock -= q;
12                 System.out.println("Nouveau stock: " + stock);
13             } catch (StockInsufficientException e) {
14                 System.out.println("Stock insuffisant");
15             }
16         }
17     }
18
19     public static void main(String[] args) {
20         new Produit().retirer(10);
21     }
22 }
```

## Affichage

A screenshot of a terminal window with a black background. The text "Stock insuffisant" is displayed in a large, white, pixelated font. The text is centered and occupies most of the width of the terminal.

## Exercice 7 (Téléchargement)

### Objectif

Gérer un téléchargement en limitant la taille de fichier via QuotaDepasseException.

## Code

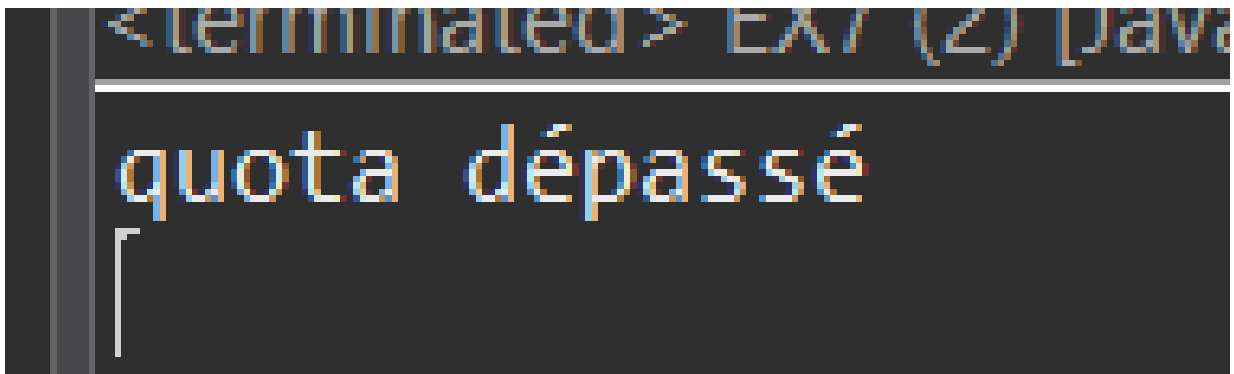
### Classe EX7

```

1 package E2;
2
3 class QuotaDepasseException extends Exception {}
4
5 public class EX7 {
6     static void telecharger(double taille) {
7         try {
8             if (taille > 100) throw new QuotaDepasseException();
9             System.out.println("téléchargement OK");
10        } catch (QuotaDepasseException e) {
11            System.out.println("quota dépassé");
12        }
13    }
14
15    public static void main(String[] args) {
16        telecharger(150);
17    }
18 }

```

## Affichage



## Exercice 8 (Validation de formulaire)

### Objectif

Créer une exception ChampObligatoireException pour valider les formulaires.

### Code

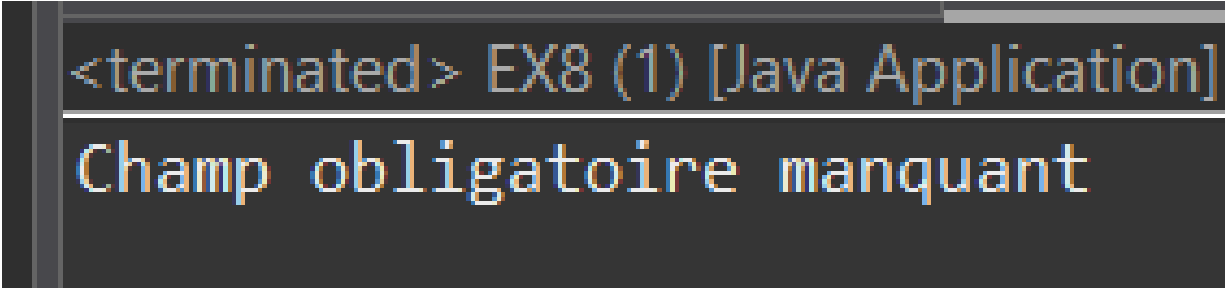
Classe EX8

```

1 package E2;
2
3 class ChampObligatoireException extends Exception {}
4
5 public class EX8 {
6     static void valider(String nom, String email) {
7         try {
8             if (nom == null || nom.isEmpty() || email == null || email.isEmpty())
9                 throw new ChampObligatoireException();
10            System.out.println("Formulaire valide");
11        } catch (ChampObligatoireException e) {
12            System.out.println("Champ obligatoire manquant");
13        }
14    }
15
16    public static void main(String[] args) {
17        valider("", "a@b");
18    }
19 }

```

## Affichage



The screenshot shows a terminal window with a dark background. The title bar at the top reads "<terminated> EX8 (1) [Java Application]". The main content area displays the text "Champ obligatoire manquant" in a light-colored, monospaced font.

## Exercice 9 (Paiement)

### Objectif

Gérer PaiementRefuseException et CarteExpireeException lors d'un paiement.

### Code

#### Classe EX9

```

1 package E2;
2
3 class PaiementRefuseException extends Exception {}
4 class CarteExpireeException extends Exception {}

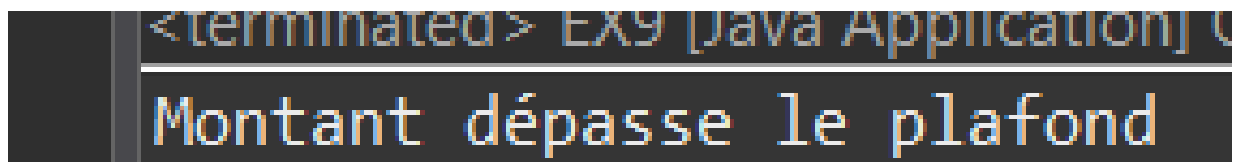
```

```

5
6 public class EX9 {
7     static void payer(double montant, boolean expiree, double plafond) {
8         try {
9             if (expiree) throw new CarteExpireeException();
10            if (montant > plafond) throw new PaiementRefuseException();
11            System.out.println("Paiement accepté");
12        } catch (CarteExpireeException e) {
13            System.out.println("Carte expirée");
14        } catch (PaiementRefuseException e) {
15            System.out.println("Montant dépasse le plafond");
16        }
17    }
18
19    public static void main(String[] args) {
20        payer(1200, false, 1000);
21    }
22 }

```

## Affichage



```

<terminated> EX9 [Java Application]
Montant dépasse le plafond

```

## Partie 3 — Généricité

### Exercice 1

#### Objectif

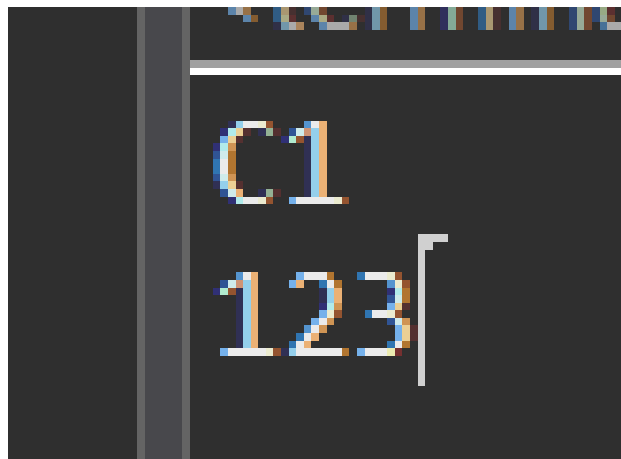
Créer une classe générique `Boite<T>` et la tester avec `String` et `Integer`.

#### Code

##### Classe EX1

```
1 package E3;
2
3 class Boite<T> {
4     private T contenu;
5     void setContenu(T c) {
6         contenu = c; }
7     T getContenu() {
8         return contenu; }
9 }
10
11 public class EX1 {
12     public static void main(String[] args) {
13         Boite<String> sb = new Boite<>();
14         sb.setContenu("C1");
15         System.out.println(sb.getContenu());
16
17         Boite<Integer> ib = new Boite<>();
18         ib.setContenu(123);
19         System.out.println(ib.getContenu());
20     }
21 }
```

## Affichage



## Exercice 2

### Objectif

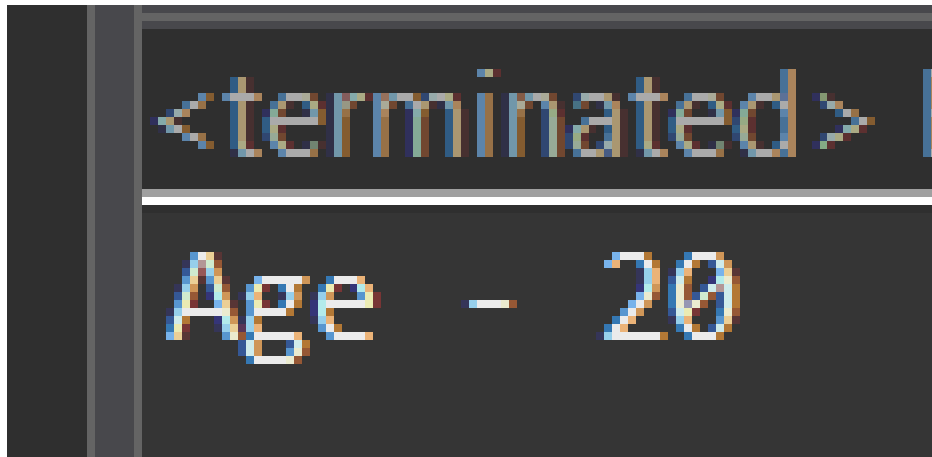
Créer une classe générique `Paire<T, U>` pour contenir deux objets.

### Code

#### Classe EX2

```
1 package E3;
2
3 class Paire<T, U> {
4     T premier;
5     U second;
6     Paire(T p, U s) {
7         premier = p; second = s;
8     }
9     void afficherPaire() { System.out.println(premier + " - " + second); }
10 }
11
12 public class EX2 {
13     public static void main(String[] args) {
14         Paire<String, Integer> p = new Paire<>("Age", 20);
15         p.afficherPaire();
16     }
17 }
```

## Affichage



## Exercice 3

### Objectif

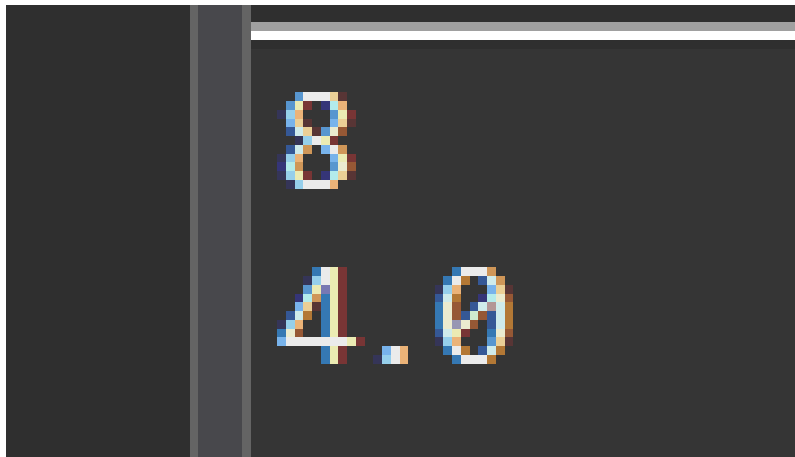
Créer une interface générique `Calcul<T>` et l'implémenter pour `Integer` et `Double`.

## Code

### Interface EX3

```
1 package E3;
2
3 interface Calcul<T> {
4     T addition(T a, T b);
5 }
6
7 class CalculInteger implements Calcul<Integer> {
8     public Integer addition(Integer a, Integer b) { return a + b; }
9 }
10
11 class CalculDouble implements Calcul<Double> {
12     public Double addition(Double a, Double b) { return a + b; }
13 }
14
15 public class EX3 {
16     public static void main(String[] args) {
17         CalculInteger ci = new CalculInteger();
18         System.out.println(ci.addition(5, 3));
19         CalculDouble cd = new CalculDouble();
20         System.out.println(cd.addition(2.5, 1.5));
21     }
22 }
```

## Affichage



## Exercice 4

### Objectif

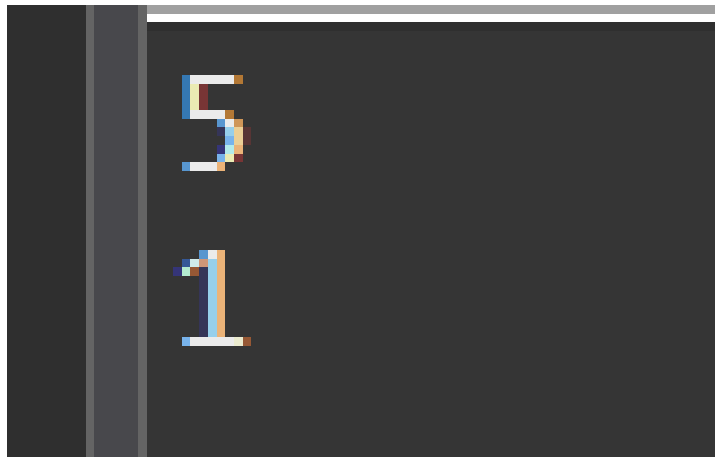
Créer une interface générique `Compareur<T>` et l'implémenter pour `Integer` et `String`.

## Code

### Interface EX4

```
1 package E3;
2
3 interface Compareur<T> {
4     int comparer(T a, T b);
5 }
6
7 class CompareurInteger implements Compareur<Integer> {
8     public int comparer(Integer a, Integer b) { return a - b; }
9 }
10
11 class CompareurString implements Compareur<String> {
12     public int comparer(String a, String b) { return a.length() - b.length(); }
13 }
14
15 public class EX4 {
16     public static void main(String[] args) {
17         CompareurInteger ci = new CompareurInteger();
18         System.out.println(ci.comparer(10, 5));
19         CompareurString cs = new CompareurString();
20         System.out.println(cs.comparer("abc", "ab"));
21     }
22 }
```

## Affichage



## Exercice 6

### Objectif

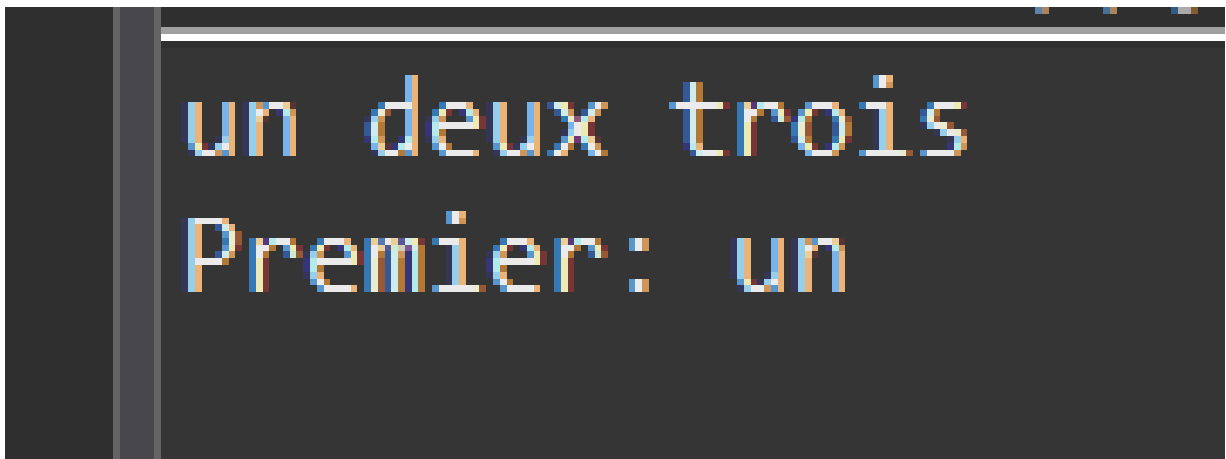
Créer des méthodes génériques : `afficherTableau(T[])` et `getPremier(T[])`.

## Code

### Classe EX6

```
1 package E3;
2
3 public class EX6 {
4     public static <T> void afficherTableau(T[] tableau) {
5         for (T e : tableau) System.out.print(e + " ");
6         System.out.println();
7     }
8
9     public static <T> T getPremier(T[] tableau) {
10         return tableau[0];
11     }
12
13     public static void main(String[] args) {
14         String[] mots = {"un", "deux", "trois"};
15         afficherTableau(mots);
16         System.out.println("Premier: " + getPremier(mots));
17     }
18 }
```

## Affichage



## Exercice 7

### Objectif

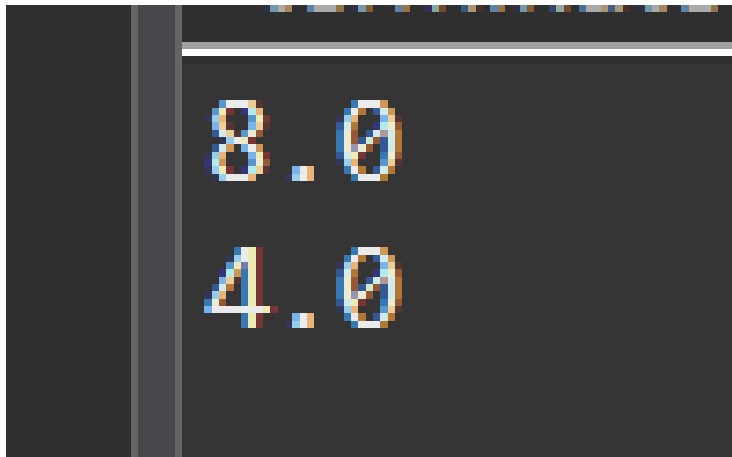
Créer une méthode générique bornée : `somme(T a, T b)`.

### Code

#### Classe EX7

```
1 package E3;
2
3 public class EX7 {
4     public static <T extends Number> double somme(T a, T b) {
5         return a.doubleValue() + b.doubleValue();
6     }
7
8     public static void main(String[] args) {
9         System.out.println(somme(5, 3));
10        System.out.println(somme(2.5, 1.5));
11    }
12 }
```

## Affichage



## Exercice 8

### Objectif

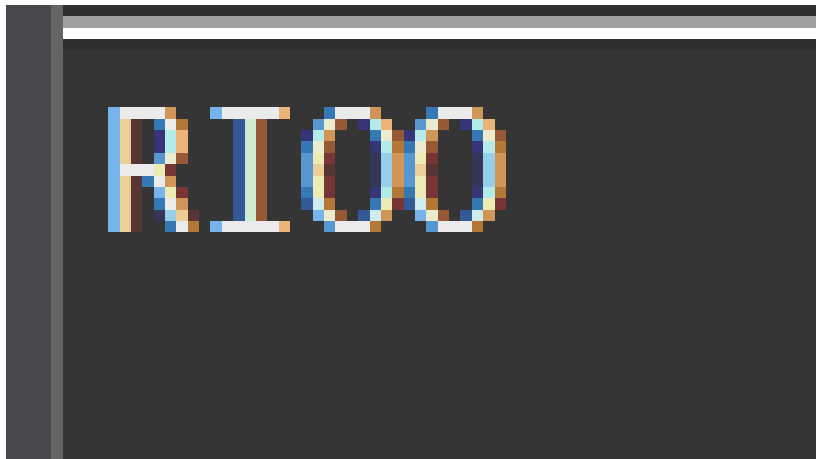
Hériter d'une classe générique avec un type spécifique (Chien extends Animal<String>).

## Code

### Classe EX8

```
1 package E3;
2
3 class Animal<T> {
4     T nom;
5     Animal(T n) { nom = n; }
6     void afficher() { System.out.println(nom); }
7 }
8
9 class Chien extends Animal<String> {
10     Chien(String n) { super(n); }
11 }
12
13 public class EX8 {
14     public static void main(String[] args) {
15         Chien c = new Chien("RIOO");
16         c.afficher();
17     }
18 }
```

## Affichage



## Exercice 9

### Objectif

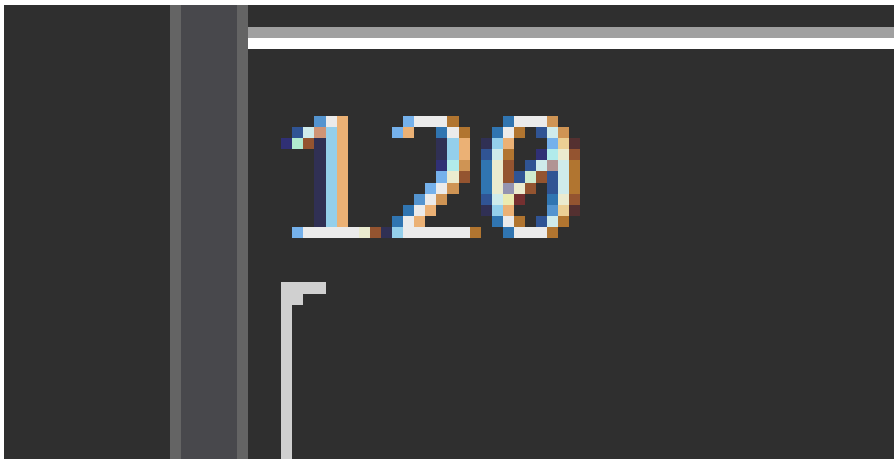
Hériter d'une classe générique en conservant la généricité (`Voiture<T> extends Vehicule<T>`).

### Code

#### Classe EX9

```
1 package E3;
2
3 class Vehicule<T> {
4     T vitesse;
5     Vehicule(T v) { vitesse = v; }
6     void afficher() { System.out.println(vitesse); }
7 }
8
9 class Voiture<T> extends Vehicule<T> {
10     Voiture(T v) { super(v); }
11 }
12
13 public class EX9 {
14     public static void main(String[] args) {
15         Voiture<Integer> v = new Voiture<>(120);
16         v.afficher();
17     }
18 }
```

## Affichage



## Exercice 10

### Objectif

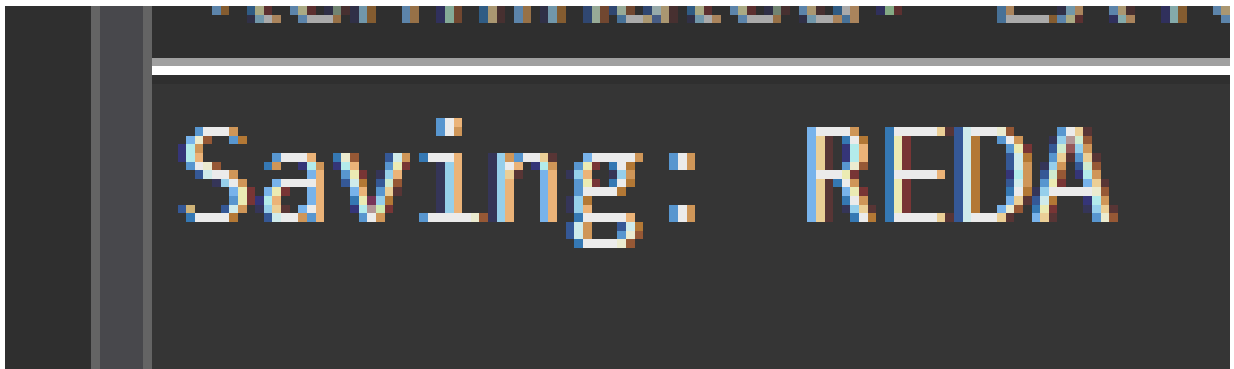
Appliquer l'héritage spécifique d'une classe générique (UserRepository extends Repository<User>).

## Code

### Classe EX10

```
1 package E3;
2
3 class User {
4     String name;
5     User(String n) { name = n; }
6     public String toString() { return name; }
7 }
8
9 class Repository<T> {
10     void save(T obj) { System.out.println("Saving: " + obj); }
11 }
12
13 class UserRepository extends Repository<User> {}
14
15 public class EX10 {
16     public static void main(String[] args) {
17         UserRepository ur = new UserRepository();
18         ur.save(new User("REDA"));
19     }
20 }
```

## Affichage



## Exercice 11

### Objectif

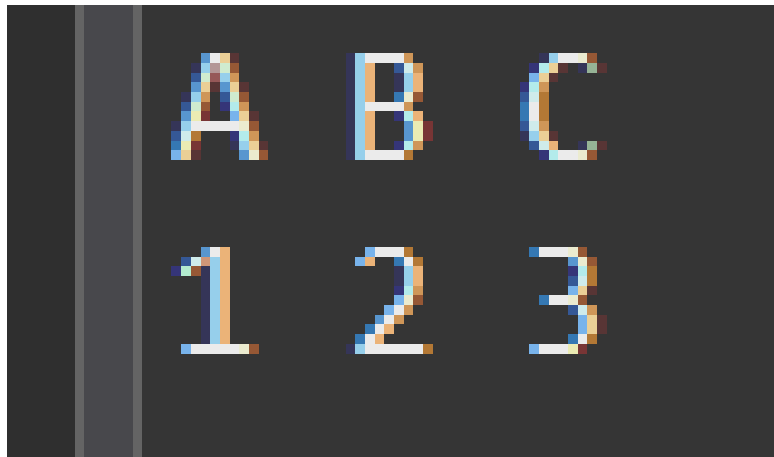
Créer une méthode en utilisant un wildcard (`?`) pour afficher une liste.

## Code

### Classe EX11

```
1 package E3;
2
3 import java.util.*;
4
5 public class EX11 {
6     static void afficherListe(List<?> liste) {
7         for (Object o : liste) System.out.print(o + " ");
8         System.out.println();
9     }
10
11     public static void main(String[] args) {
12         List<String> ls = Arrays.asList("A", "B", "C");
13         List<Integer> li = Arrays.asList(1, 2, 3);
14         afficherListe(ls);
15         afficherListe(li);
16     }
17 }
```

## Affichage



## Exercice 12

### Objectif

Créer une méthode en utilisant un wildcard borné (`? extends Number`) pour afficher des nombres.

### Code

#### Classe EX12

```
1 package E3;
2
3 import java.util.*;
4
5 public class EX12 {
6     static void afficherNombres(List<? extends Number> liste) {
7         for (Number n : liste) System.out.print(n + " ");
8         System.out.println();
9     }
10
11     public static void main(String[] args) {
12         List<Integer> li = Arrays.asList(1, 2, 3);
13         List<Double> ld = Arrays.asList(1.1, 2.2, 3.3);
14         afficherNombres(li);
15         afficherNombres(ld);
16     }
17 }
```

## Affichage

